



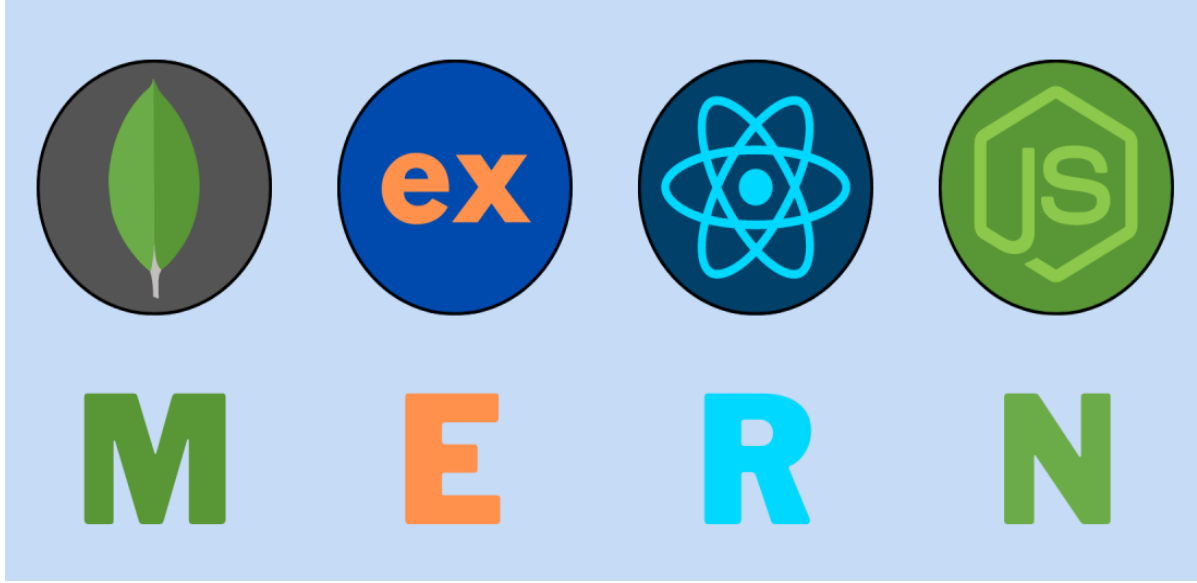
**ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ**

( ವಿ.ಟಿ.ಯು. ಅಡಿಯಲ್ಲಿನ ಸ್ವಾಯತ್ತ ಸಂಸ್ಥೆ )

**BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT**  
(Autonomous Under VTU)

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE  
AND MACHINE LEARNING**

**LAB MANUAL**



**Subject Name: MERN DB LAB**

**Course Code: BAIL456C**

**IV Semester  
Laboratory Manual**

**Prepared By,**

**Dr. Archana Bhat**

**Assistant Professor**

**Department of AI and ML BMSIT and M**

## **VISION**

**To emerge as a leading department in AI and ML by preparing skilled, responsible, and eco-friendly professionals who use technology to improve society.**

## **MISSION**

- 1.To equip students with in-depth knowledge in Artificial Intelligence and Machine Learning, built upon a robust foundation in Computer Science and Engineering through industry focused curriculum with practical learning.**
- 2. To encourage research and innovation through industry partnerships and sustainable technology practices.**
- 3. To inspire students toward ethical leadership and entrepreneurship through innovative and collaborative student-led activities.**

### **PROGRAMME EDUCATIONAL OBJECTIVES (PEO'S)**

**Graduates will thrive as IT professionals, applying AI and ML to build creative and lasting solutions.**

**Graduates will pursue further studies, research, and entrepreneurship, keeping up with technological advancements in various fields.**

**Graduates will demonstrate ethics, integrity, leadership, teamwork, and a dedication to continuous learning in their careers.**

### **PROGRAMME SPECIFIC OUTCOMES (PSO)**

**Apply foundational knowledge of computer science along with AI and ML techniques to design effective, real-world solutions across diverse application domains.**

**Develop AI-Driven innovation with a commitment to ethics, social responsibility and sustainability.**

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**  
**Choice Based Credit System (CBCS)**  
**SEMESTER – IV**

**MERN (0:0:2:0) 1**  
(Effective from the academic year 2023-2024)

|                                |                 |            |    |
|--------------------------------|-----------------|------------|----|
| Course Code                    | <b>BAIL456C</b> | CIE Marks  | 50 |
| Teaching Hours/Week (L:T:P: S) | 0:0:2:0         | SEE Marks  | 50 |
| Credits                        | 01              | Exam Hours | 02 |
| Examination type (SEE)         | Practical       |            |    |

**Course Objectives:**

This course aims to prepare the students to:

- Understand and apply critical web development languages and tools to create dynamic and responsive web applications.
- To build server-side applications using Node.js and Express
- Develop user interfaces with React.js,
- Manage data using MongoDB, and integrate these technologies to create full stack apps
- Understanding APIs and routing.

| Sl.NO | Experiments                                                                                                                                                                                                                                                                                                                                                                                     |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | a. Using MongoDB, create a collection called transactions in database usermanaged (drop if it already exists) and bulk load the data from a json file, transactions.json<br>c. Upsert the record from the new file called transactions_upsert.json in Mongoddb.                                                                                                                                 |
| 2     | Query MongoDB with Conditions: [Create appropriate collection with necessary documents to answer the query]<br>a. Find any record where Name is Somu<br>b. Find any record where total payment amount (Payment.Total) is 600.<br>c. Find any record where price (Transaction.Price) is between 300 to 500.<br>Calculate the total transaction amount by adding up Payment.Total in all records. |
| 3     | a. Write a program to check request header for cookies.<br>b. write node.js program to print the car object properties, delete the second property and get length of the object.                                                                                                                                                                                                                |
| 4     | a. Read the data of a student containing usn, name, sem, year_of_admission from node js and store it in the mongoddb.<br>b. For a partial name given in node js, search all the names from mongoddb student documents created in Question(a)                                                                                                                                                    |
| 5     | Implement all CRUD operations on a File System using Node JS                                                                                                                                                                                                                                                                                                                                    |
| 6     | Develop the application which takes name and email data from HTML form index.html after submission and sends the data in the form of object from client side to Node.js server using Ajax.                                                                                                                                                                                                      |
| 7     | Develop an authentication mechanism with email_id and password using HTML and Express JS (POST method)                                                                                                                                                                                                                                                                                          |
| 8     | Demonstrate Express JS routing mechanism by developing two routes: User and Admin route. User route returns some user details and Admin route returns some admin details.                                                                                                                                                                                                                       |
| 9     | Develop a React code to build a simple search filter functionality to display a filtered list based on the search query entered by the user.                                                                                                                                                                                                                                                    |
| 10    | Develop a React code to collect data from rest API.                                                                                                                                                                                                                                                                                                                                             |

**Course outcomes (Course Skill Set):**

At the end of the course the student will be able to:

CO1: Apply the fundamentals of MongoDB, such as data modelling, CRUD operations, and basic queries to solve given problem.

CO2: Use constructs of Express.js, including routing, software and constructing RESTful APIs to solve real world problems.

CO3: Develop scalable and efficient RESTful APIs using NodeJS.

CO4: Develop applications using React, including components, state, props, and JSX syntax.

**Suggested Learning Resources:**

- Vasan Subramanian Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node, Apress; 1st ed. edition (1 April 2017)
- Eddy Wilson Iriarte Koroliova, MERN Quick Start Guide, Packt Publishing (31 May 2018),
- <https://www.geeksforgeeks.org/mern-stack/>
- <https://blog.logrocket.com/mern-stack-tutorial/>

## MERN Course Objectives

- Understand and apply critical web development languages and tools to create dynamic and responsive web applications.
  - To build server-side applications using Node.js and Express.
  - Develop user interfaces with React.js.
  - Manage data using MongoDB, and integrate these technologies to create full stack apps.
  - Understanding APIs and routing.
-

## LAB PROGRAMS

### Experiment 1: MongoDB Bulk Load and Upsert

**Objective:** \* Using MongoDB, create a collection called transactions in database usermanaged (drop if it already exists) and bulk load the data from a json file, transactions.json.

- Upsert the record from the new file called transactions\_upsert.json in MongoDB.

#### Solution (MongoDB Shell / Command Line):

1. **Bulk Load:** Run this in your terminal to drop the collection (if it exists) and import data.

Bash

```
mongo usermanaged --eval "db.transactions.drop()"
mongoimport --db usermanaged --collection transactions --file
transactions.json --jsonArray
```

2. **Upsert:** To upsert data using a JSON file, use mongoimport with the --mode=upsert flag (requires matching \_id fields).

Bash

```
mongoimport --db usermanaged --collection transactions --file
transactions_upsert.json --jsonArray --mode=upsert
```

---

## Experiment 2: Querying MongoDB with Conditions

**Objective:** Query MongoDB with Conditions: [Create appropriate collection with necessary documents to answer the query]

- Find any record where Name is Somu.
- Find any record where total payment amount (Payment.Total) is 600.
- Find any record where price (Transaction.Price) is between 300 to 500.
- Calculate the total transaction amount by adding up Payment.Total in all records.

### Solution (MongoDB Shell):

#### JavaScript

```
// 1. Setup sample data
use usermanaged;
db.sales.insertMany([
  { Name: "Somu", Payment: { Total: 600 }, Transaction: { Price: 400 } },
  { Name: "Alice", Payment: { Total: 800 }, Transaction: { Price: 350 } },
  { Name: "Bob", Payment: { Total: 200 }, Transaction: { Price: 200 } }
]);
```

```
// 2. Find any record where Name is Somu
db.sales.find({ Name: "Somu" });
```

```
// 3. Find any record where total payment amount is 600
db.sales.find({ "Payment.Total": 600 });
```

```
// 4. Find any record where price is between 300 to 500
db.sales.find({ "Transaction.Price": { $gte: 300, $lte: 500 } });
```

```
// 5. Calculate the total transaction amount
db.sales.aggregate([
  { $group: { _id: null, TotalAmount: { $sum: "$Payment.Total" } } }
]);
```

---

### Experiment 3: Node.js Basics (Cookies & Objects)

**Objective 3a:** Write a program to check request header for cookies.

**Objective 3b:** Write node.js program to print the car object properties, delete the second property and get length of the object.

**Solution 3a (checkCookies.js):**

JavaScript

```
const http = require('http');

http.createServer((req, res) => {
  const cookies = req.headers.cookie;
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end(`Cookies from request: ${cookies || 'No cookies found'}`);
}).listen(3000, () => console.log('Server running on port 3000'));
```

**Solution 3b (carObject.js):**

JavaScript

```
let car = { make: "Toyota", model: "Corolla", year: 2022 };

console.log("Original Properties:", Object.keys(car));
delete car.model;
console.log("After Deletion:", car);
console.log("Length of object:", Object.keys(car).length);
```



## Experiment 4: Node.js to MongoDB Operations

**Objective 4a:** Read the data of a student containing usn, name, sem, year\_of\_admission from node js and store it in the mongodb.

**Objective 4b:** For a partial name given in node js, search all the names from mongodb student documents created in Question(a).

### Solution (studentDb.js):

JavaScript

```
const { MongoClient } = require('mongodb');
const uri = "mongodb://localhost:27017";
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    const db = client.db('college');
    const students = db.collection('students');

    // Part A: Store Student
    const studentData = { usn: "IBM20CS001", name: "Rahul Kumar", sem: 4,
year_of_admission: 2020 };
    await students.insertOne(studentData);
    console.log("Student added.");
    // Part B: Search Partial Name (e.g., "Rah")
    const partialName = "Rah";
    const results = await students.find({ name: { $regex: partialName, $options: "i"
} }).toArray();
    console.log("Search Results:", results);
  } finally {
    await client.close();
  }
}
run().catch(console.dir);
```

---

## Experiment 5: Node JS File System CRUD

**Objective:** Implement all CRUD operations on a File System using Node JS.

**Solution (fsCrud.js):**

JavaScript

```
const fs = require('fs');  
const filename = 'data.txt';
```

// Create / Update

```
fs.writeFileSync(filename, 'Hello, this is the initial text.\n');  
console.log("File Created.");
```

// Read

```
let content = fs.readFileSync(filename, 'utf8');  
console.log("Read Data:", content);
```

// Update (Append)

```
fs.appendFileSync(filename, 'This is appended text.');
```

```
console.log("File Updated.");
```

// Delete

```
fs.unlinkSync(filename);  
console.log("File Deleted.");
```

---

## Experiment 6: Node.js & AJAX

**Objective:** Develop the application which takes name and email data from HTML form index.html after submission and sends the data in the form of object from client side to Node.js server using Ajax.

### Solution:

#### index.html:

##### HTML

```
<form id="myForm">
  Name: <input type="text" id="name"><br>
  Email: <input type="text" id="email"><br>
  <button type="button" onclick="sendData()">Submit</button>
</form>
```

```
<script>
function sendData() {
  const data = {
    name: document.getElementById('name').value,
    email: document.getElementById('email').value
  };
  fetch('/submit', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  }).then(res => res.text()).then(alert);
}
</script>
```

#### server.js:

##### JavaScript

```

const express = require('express');
const app = express();
app.use(express.json());
app.use(express.static(__dirname));

app.post('/submit', (req, res) => {
  console.log("Received Data:", req.body);
  res.send("Data received successfully via Ajax!");
});

app.listen(3000, () => console.log('Server running on 3000'));

```

---

## Experiment 7: Express Authentication Form

**Objective:** Develop an authentication mechanism with email\_id and password using HTML and Express JS (POST method).

**Solution (authServer.js):**

JavaScript

```

const express = require('express');
const app = express();

app.use(express.urlencoded({ extended: true }));

app.get('/', (req, res) => {
  res.send(`
    <form action="/login" method="POST">
      Email: <input type="email" name="email_id" required><br>
      Password: <input type="password" name="password" required><br>
      <button type="submit">Login</button>
    </form>
  `);
});

app.post('/login', (req, res) => {

```

```

const { email_id, password } = req.body;
if (email_id === "admin@test.com" && password === "1234") {
  res.send("Authentication Successful");
} else {
  res.send("Invalid Credentials");
}
});

app.listen(3000, () => console.log("Server running on port 3000"));

```

---

## Experiment 8: Express Routing Mechanism

**Objective:** \* Demonstrate Express JS routing mechanism by developing two routes: User and Admin route.

- User route returns some user details and Admin route returns some admin details.

### Solution (routes.js):

#### JavaScript

```

const express = require('express');
const app = express();

// User Route
app.get('/user', (req, res) => {
  res.json({ role: "User", name: "John Doe", status: "Active" });
});

// Admin Route
app.get('/admin', (req, res) => {
  res.json({ role: "Admin", name: "Super User", privileges: ["read", "write", "delete"] });
});

app.listen(3000, () => console.log('Routing server listening on port 3000'));

```

---

## Experiment 9: React Search Filter

**Objective:** Develop a React code to build a simple search filter functionality to display a filtered list based on the search query entered by the user.

### Solution (App.jsx):

JavaScript

```
import React, { useState } from 'react';

function App() {
  const [query, setQuery] = useState("");
  const items = ["Apple", "Banana", "Orange", "Mango", "Grapes"];

  const filteredItems = items.filter(item =>
    item.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <div>
      <h2>Search Filter</h2>
      <input
        type="text"
        placeholder="Search fruits..."
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      <ul>
        {filteredItems.map((item, index) => <li key={index}>{item}</li>)}
      </ul>
    </div>
  );
}

export default App;
```

---

## Experiment 10: React Data Fetching from REST API

**Objective:** Develop a React code to collect data from rest API.

### Solution (App.jsx):

JavaScript

```
import React, { useState, useEffect } from 'react';

function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(response => response.json())
      .then(data => setUsers(data))
      .catch(error => console.error("Error fetching data:", error));
  }, []);

  return (
    <div>
      <h2>User List from API</h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>{user.name} - {user.email}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

---

## Annexure: System Environment Setup (Ubuntu/Debian Linux)

To execute the experiments above, prepare your Linux environment by installing the necessary databases, runtimes, and framework dependencies via the command line.

### 1. Installing Node.js & npm (via NVM)

Using the Node Version Manager (NVM) is the recommended method for developers, as it allows you to easily switch between Node.js versions and avoids global permission errors.

- **Download and install NVM:**

Bash

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh |  
bash
```

- **Reload your shell configuration:**

Bash

```
source ~/.bashrc
```

- **Install the latest LTS version of Node.js:**

Bash

```
nvm install --lts
```

- **Verify installation:**

Bash

```
node -v
```

```
npm -v
```

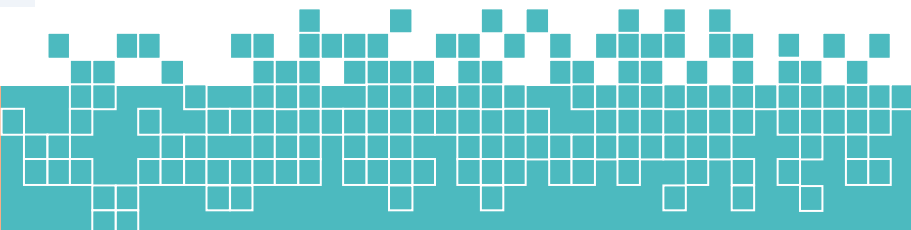
### 2. Installing MongoDB Community Edition

- **Install prerequisites:**

Bash

```
sudo apt-get update
```

```
sudo apt-get install gnupg curl
```





- **Import the MongoDB public GPG key:**

Bash

```
curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc | sudo gpg -o  
/usr/share/keyrings/mongodb-server-8.0.gpg --dearmor
```

- **Create the list file (example for Ubuntu 24.04 Noble):**

Bash

```
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ]  
https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 multiverse" | sudo tee  
/etc/apt/sources.list.d/mongodb-org-8.0.list
```

- **Update the package database and install MongoDB:**

Bash

```
sudo apt-get update  
sudo apt-get install -y mongodb-org
```

- **Start and enable the MongoDB daemon:**

Bash

```
sudo systemctl start mongod  
sudo systemctl enable mongod
```

### 3. Setting Up Project Dependencies

For experiments requiring an Express server or MongoDB connection (Experiments 4, 6, 7, and 8), initialize a project directory and install the specific Node packages.

- **Initialize a Node project:**

Bash

```
mkdir mern-lab && cd mern-lab  
npm init -y
```

- **Install Express and the MongoDB Driver:**

Bash

```
npm install express mongodb
```

#### 4. Setting Up React (Experiments 9 & 10)

Modern React applications are typically scaffolded using tools like Vite.

- **Create a new React project:**

Bash

```
npm create vite@latest my-react-app -- --template react
```

- **Navigate into the directory and install dependencies:**

Bash

```
cd my-react-app
```

```
npm install
```

- **Start the React development server:**

Bash

```
npm run dev
```

Would you like me to walk you through how to execute any of these individual script files via the terminal now that your environment setup steps are documented?

**Thank  
You**

